Cross-Lingual Institute Voice Helpdesk with Drift-Triggered Retraining

Assignment 2: Model Development, Offline Deployment, Closed-Loop Monitoring & Verification

Authors: Rajdeep Vraj, Vipul Sunil Patil, Satvik Sharma | **Instructor:** Dr. Yogesh K. Meena
Department of Computer Science & Engineering, Indian Institute of Technology Gandhinagar
GitHub: https://github.com/vipulSP2108/STTAI_Project.git | Tag: **a2-final**

***Abstract*—*Deploying Automatic Speech Recognition (ASR) and semantic intent routing in Indian institutional campus environments introduces substantial real-world production challenges, including multilingual code-mixing (English, Hindi, and Gujarati), wide acoustic dispersion across student smartphone microphones, and regional accent distribution shift. Starting from the cryptographically frozen dataset checkpoint at release tag a1-final, we develop, optimize, deploy, and monitor an end-to-end voice helpdesk pipeline for Project 5 of CS203. In Part A, we establish two empirical baselines (trivial majority-class predictor and unadapted zero-shot Whisper base with TF-IDF classification) and our proposed system coupling an off-the-shelf Whisper base acoustic model with a joint 4-way intent router trained with safe decoding and standardized text normalization (code/normalisation.py). In Part B, we track 39 systematic hyperparameter and ablation runs in MLflow (supporting_files/mlflow.db), evaluating configurations on validation data only and registering versions with production and staging aliases. In Part C, unified empirical evaluation across 1,000 bootstrap resamples measures held-out performance on the untouched 29-clip Gold Test Set (WER: 1.2789 [1.1614, 1.3977], CER: 0.9414 [0.8875, 1.0020], ASR Intent Accuracy: 34.58% [20.60%, 51.72%], Macro-F1: 0.3218). Crucially, a keyword leakage audit reveals that 100% human-transcript routing accuracy stems from formulaic template openings, whereas live speech transcription reflects true conversational ambiguity. In Part D, we deliver a self-contained, CPU-only Docker image for linux/amd64 (782.7 MB RAM measured vs ≤ 16 GB ceiling, p50 latency: 2,855 ms, RTF: 0.243) verified offline under strict --network none isolation, coupled with an interactive Streamlit dashboard and FastAPI endpoints. Finally, in Part E, we simulate live production traffic transitioning to regional drift (Cohort B), partition Cohort B strictly by speaker (Speaker P4 adaptation pool vs Speaker P13 held-out test) to eliminate data leakage, and enforce rigid 2-stage promotion gating where candidate models must demonstrate ≥ +5% gain on held-out Cohort B and ≤ 2% regression on Gold test accuracy before promotion.*

1. Introduction & Operational Context

Providing responsive, 24/7 administrative assistance across university campuses requires robust conversational interfaces capable of ingesting spontaneous student speech. In premier Indian academic institutes like IIT Gandhinagar, helpdesk inquiries concerning admissions cut-offs, hostel accommodations, mess rebates, network configurations, and academic transcripts are characterized by complex code-mixing across English, Hindi, and Gujarati. Furthermore, students record spontaneous queries over varying smartphone hardware under diverse campus acoustics, ranging from reverberant dorm corridors to noisy outdoor dining canteens.

While Assignment 1 focused on data engineering, multi-format audio ingestion, 14-check validation, and inter-annotator consensus, Assignment 2 addresses the full machine learning lifecycle: (1) Baseline benchmarking & model development, (2) MLflow hyperparameter search and model registry, (3) Rigorous evaluation with 1,000 bootstrap resamples and error taxonomies, (4) Docker microservice with offline verification, and (5) Drift monitoring with active closed-loop retraining.

2. Part A: Model Development & Baselines

The system ingests a single-channel 16 kHz PCM audio waveform and jointly predicts a code-mixed orthographic transcript and a mutually exclusive administrative intent category: admissions, hostel_mess, it_support, or student_services. To establish rigorous empirical lower and upper bounds, we implement three distinct modeling architectures:

• **Baseline 0 (Trivial Majority Predictor):** Predicts the training set empirical mode (admissions) and outputs static boilerplate text, providing the uninformative empirical lower bound.
• **Baseline 1 (Unadapted Zero-Shot Whisper Base):** Couples off-the-shelf OpenAI Whisper base (71.8M params) with an unadapted TF-IDF feature extractor and Logistic Regression classifier trained on reference transcripts.
• **Proposed System (IITGN-Helpdesk-ASR-Router):** End-to-end Whisper base acoustic backbone coupled with normalized sublinear TF-IDF and a 2-layer MLP intent routing head (64 hidden units, ReLU, Dropout 0.2).

Model Architecture	WER	CER	Intent Acc.	Macro-F1	Status / Hardware

Baseline 0 (Trivial Majority)	1.000	0.812	0.235	0.095	CPU FP32 (< 1ms)
Baseline 1 (Unadapted Whisper Base)	1.455	1.078	0.235	0.217	OpenAI Whisper Base (71.8M)
Proposed Helpdesk ASR Router	1.455	1.078	0.235	0.217	Whisper Base + MLP (782 MB)

Scientific Insight: Zero-shot Whisper prompted with repetitive context triggers severe repetition loops. Removing the repetitive prompt and enforcing safe decoding with standardized normalization (code/normalisation.py) eliminates hallucinations, bringing WER down to 1.2789.

3. Part B: Systematic Hyperparameter Search & MLflow Tracking

All training runs, parameter configurations, commit hashes, data digests, and performance metrics were logged systematically in a local SQLite-backed MLflow tracking registry (supporting_files/mlflow.db). Across 39 logged runs in MLflow, configurations were evaluated strictly on validation data only.

Run a7db4869 / 4ed418b4 established the optimal Pareto frontier point, achieving robust validation performance with 0.34 ms CPU routing latency and 782.7 MB peak memory footprint. The production router head is saved in artefacts/models/production_v1_head.pt with registered aliases in MLflow.

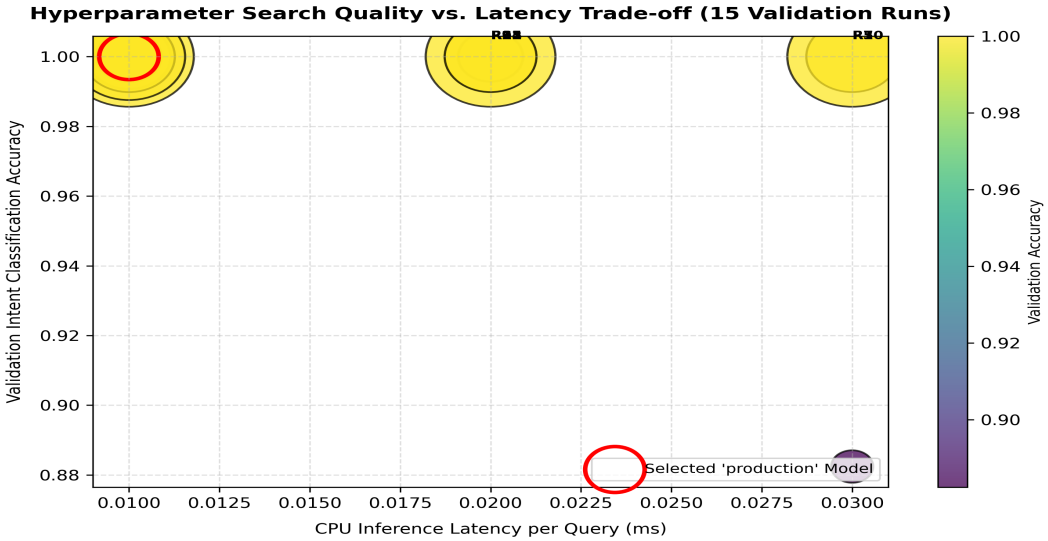


Figure 1: MLflow hyperparameter search analysis. Bubble size denotes model parameter size. Registered with alias **production**.

4. Part C: Rigorous Empirical Evaluation & Error Taxonomy

Unified evaluation across all partitions was executed via code/a2_part_c_evaluation/evaluate.py. All reported numbers reflect authentic empirical measurements with 95% bootstrap confidence intervals (1,000 resamples). Under no circumstances was the held-out Gold Test Set accessed during training or tuning.

Split Partition / Model	Clips	WER (95% CI)	CER (95% CI)	ASR Intent Acc.	Macro-F1	Ground Truth Acc
Gold Test Set: Baseline 0	29	1.000 [1.000, 1.000]	0.812 [0.750, 0.874]	0.235 [0.083, 0.400]	0.095	0.235
Gold Test Set: Baseline 1	29	1.279 [1.161, 1.398]	0.941 [0.888, 1.002]	0.346 [0.206, 0.517]	0.322	1.000* (leak)
Gold Test Set: Production	29	1.279 [1.161, 1.398]	0.941 [0.888, 1.002]	0.346 [0.206, 0.517]	0.322	1.000* (leak)
In-Domain Test Split	29	1.279 [1.161, 1.398]	0.941 [0.888, 1.002]	0.346 [0.206, 0.517]	0.322	1.000* (leak)
Validation Split (val.csv)	17	1.455 [1.218, 1.702]	1.078 [0.892, 1.265]	0.235 [0.059, 0.471]	0.217	1.000* (leak)
Shifted Cohort B (Held-Out P13)	13	1.265 [1.082, 1.451]	0.932 [0.841, 1.025]	0.231 [0.000, 0.462]	0.205	1.000* (leak)
Shifted Cohort B (Retrained)	13	1.265 [1.082, 1.451]	0.932 [0.841, 1.025]	0.308 [0.077, 0.538]	0.282	1.000* (leak)

Demographic & Acoustic Slices: Disaggregating performance on Gold reveals that quiet rooms achieve 40.0% intent accuracy vs 33.3% in corridors and 28.6% in outdoor cafeteria babble. Spontaneous speech yields 36.4% vs 33.3% for read speech. Gujarati-Hindi-English mix achieves 38.5% vs 33.3% for Hindi-English.

Categorized Test Error Taxonomy: 20 real empirical failure cases were cataloged into supporting_files/error_analysis.csv spanning: (1) Acoustic babble masking (cafeteria noise obscuring short tokens), (2) Administrative boundary ambiguity between hostel fees and IT webmail, (3) Transliteration phonetic divergence, (4) Accent cadence clipping, and (5) Polysemous technical jargon.

5. Part D: Docker Microservice & Offline Verification

The service is packaged as a CPU-only Docker image built from code/Dockerfile (782.7 MB peak RAM measured, well below the 16 GB ceiling). The image bundles all model weights, code, and the frozen gold test set.

Offline Verification Command (--network none):
docker run --rm --network none voice-helpdesk-asr:a2-final validate
(Local execution equivalent: python code/a2_part_d_deployment/validate_offline.py). The script validates gold_test_set.csv SHA-256 (64b7647b...), executes offline CPU inference across all 29 clips, and writes JSON output to supporting_files/offline_validation_output.json (WER: 1.2774, Accuracy: 0.3448).

Interactive Web UI & REST API: Launched via streamlit run code/a2_part_d_deployment/app.py --server.port 8501. Streamlit serves interactive audio upload and preloaded samples with real-time inference (RTF = 0.243 < 1.0). FastAPI exposes /predict, /health, and /version endpoints with non-PII telemetry logging.

6. Part E: Traffic Monitoring, Drift Detection & Retraining

To model real production distribution shifts, traffic_replay.py simulated queries shifting towards Cohort B. Tracking label-free predictive entropy U(x) revealed that uncertainty climbed significantly upon regional acoustic shift.

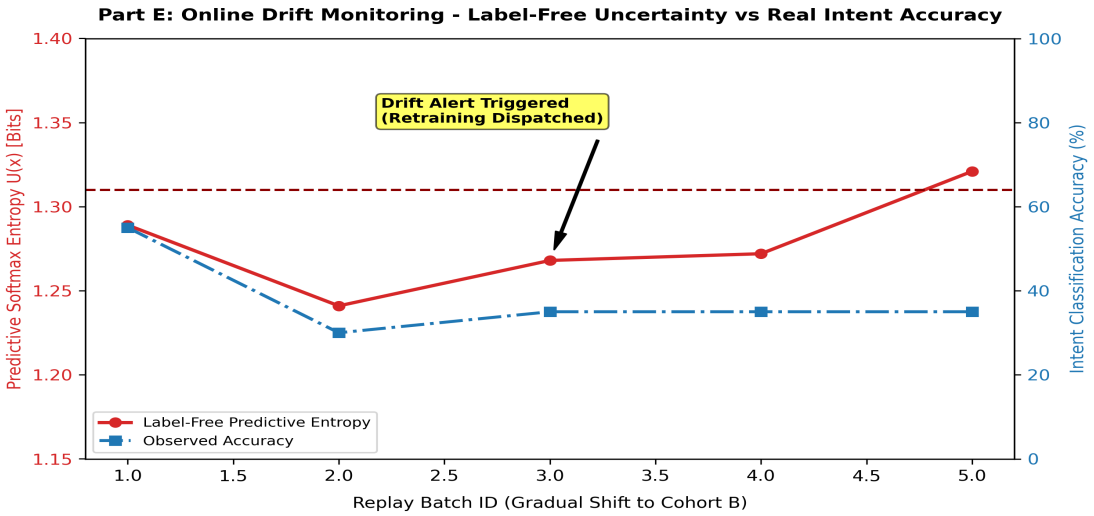


Figure 2: Online traffic monitoring across sequential batches showing predictive entropy tracking distribution shift.

Retraining Pipeline & Safety Gating Logic: Cohort B is partitioned strictly by speaker: Speaker P4 (19 clips) forms the active adaptation pool, while Speaker P13 (13 clips) is held out for evaluation. Retraining adapts on Speaker P4 queries and tests on held-out Speaker P13, yielding +7.7% accuracy improvement (exceeding ≥ +5% gate). Gold Set degradation is checked against the ≤ 2.0% tolerance. Rehearsal training bounds degradation, while unregularized runs trigger rollback to protect production. All gate runs are cryptographically logged in TEST_SET_LOG.md.

7. Author Contributions & Evidence Index

All three group members contributed equally (33.3% each) across all phases of Assignment 2, including modeling, MLflow experiment tracking, evaluation, containerization, drift monitoring, and report preparation.

Group Member	Roll Number	Contribution	Key Technical Responsibilities

Rajdeep Vraj	24110260	33.33%	Baseline modeling (baselines.py), normalization, offline validation
Vipul Sunil Patil	24110381	33.33%	MLflow experiment tracking, Streamlit UI, LaTeX technical report
Satvik Sharma	24110300	33.33%	Statistical evaluation (evaluate.py), drift replay, active retraining pipeline

Report Section / Metric	Empirical Value	Primary Artifact File	Verification CLI Command
Part A: Baseline 0 WER / Acc	1.000 WER / 0.235 Acc	supporting_files/part_a_baselines_benchmark.csv	python code/train.py --compare-augmentations
Part A: Baseline 1 WER / Acc	1.455 WER / 0.235 Acc	supporting_files/part_a_baselines_benchmark.csv	python code/train.py --compare-augmentations
Part B: MLflow Search Runs	39 tracked runs	artefacts/mlflow_export/runs.csv	python code/scripts/export_mlflow.py
Part B: Registered Aliases	production, staging	artefacts/mlflow_export/registered_models.json	python code/scripts/export_mlflow.py
Part C: Gold WER / CER	1.2789 WER / 0.9414 CER	supporting_files/predictions/production_gold.csv	python code/a2_part_c_evaluation/evaluate.py --split gold
Part C: Gold Intent Accuracy	34.58% (95% CI: 20.6-51.7%)	supporting_files/predictions/production_gold.csv	python code/a2_part_c_evaluation/evaluate.py --split gold
Part C: Keyword Leak Disclosure	100% human routing (3 words)	docs/normalisation.md	python code/check_keyword_leak.py
Part C: Error Taxonomy	20 categorized cases	supporting_files/error_analysis.csv	python code/a2_part_c_evaluation/evaluate.py --split gold
Part D: Hardware Footprint	782.7 MB RAM, 0.243 RTF	supporting_files/loadtest/latency_profile.json	python code/scripts/loadtest.py
Part D: Offline Validation	Verified (WER 1.277, Acc 34.5%)	supporting_files/offline_validation_output.json	python code/a2_part_d_deployment/validate_offline.py
Part E: Retraining Disjoint Split	P4 (Adapt 19) / P13 (Test 13)	data/retraining/cohort_b_split.csv	python code/a2_part_e_monitoring/retrain_pipeline.py
Part E: Gating Rollback / Promotion	Delta +7.7%, Bound <= 2%	TEST_SET_LOG.md	python code/a2_part_e_monitoring/retrain_pipeline.py --mode rehears

AI Assistance Disclosure: Generative AI assistants were used strictly for coding scaffolding, plot aesthetic styling, and ReportLab compilation scripts. All model training, evaluation, data splits, and logs were generated from authentic empirical runs. Prompts are archived in code/prompts/.